

Assignment IV

DATA301/DATA471

Pablo Fernandez

Exercise 1

Part 1 — Order statistics: comparison of alternatives

Summary of Wicklin (2017) and Hyndman & Fan (1996)

According to Hyndman & Fan (1996) there are nine definitions of sample quantiles that commonly appear in statistical software packages. All nine methods can be expressed through a common interpolation formula:

$$Q(p) = (1 - \gamma) x_{[j]} + \gamma x_{[j+1]}$$

where $j = \lfloor h \rfloor$, $\gamma = h - j$, and h is a real-valued index that depends on p , n , and a method-specific parameter m . The nine types split into two groups:

Discontinuous methods (Types 1–3) always return a value that is actually present in the sample (or an average of two adjacent values). They set $\gamma = 0$ or $\gamma = 1$ via a rounding rule:

- **Type 1:** Inverse of the empirical CDF; rounds up — $Q(p) = x_{[np]}$.
- **Type 2:** Average of the values to the left and right of the jump in the empirical CDF (SAS default). Returns the midpoint of adjacent observations when np is an integer.
- **Type 3:** Round to the nearest even index when np is exactly a half-integer (used by some older packages).

Continuous methods (Types 4–9) linearly interpolate between adjacent order statistics using $h = np + m(p)$:

Type	$h =$	Notes
4	np	Piecewise-linear CDF, no shift

Type	$h =$	Notes
5	$np + 0.5$	Piecewise-linear CDF with 0.5 offset
6	$(n + 1)p$	Weibull plotting positions; used by Minitab, SPSS
7	$(n - 1)p + 1$	R default ; plotting positions $(j - 1)/(n - 1)$
8	$(n + 1/3)p + 1/3$	Approximately median-unbiased for any distribution
9	$(n + 1/4)p + 3/8$	Approximately unbiased for the normal distribution (Blom)

Wicklin (2017) illustrates these differences visually and emphasises that for large samples the nine definitions converge; meaningful differences emerge mainly with small n or extreme quantiles.

Which type would I choose and why?

For this exercise, where the data are modelled as i.i.d. $\text{Exponential}(\lambda)$ which is a strongly right-skewed distribution I would choose **Type 6** (Weibull plotting positions, $h = (n + 1)p$). Hyndman & Fan show that Type 6 is *approximately median-unbiased for any continuous distribution*, not just symmetric or normal ones. Since the exponential distribution is far from symmetric, median unbiasedness over the class of all continuous distributions is a more robust guarantee than the normality-tuned Type 8 or Type 9. Type 6 also has a simple probabilistic interpretation: the k -th order statistic of a sample of size n has median at approximately $p = k/(n + 1)$, which is exactly the plotting position Type 6 inverts.

R's default

R uses **Type 7** by default in `quantile()`. This corresponds to $h = (n - 1)p + 1$, which spaces the quantile estimates uniformly between the minimum and maximum of the sample (plotting positions $(j - 1)/(n - 1)$). It is a sensible general-purpose choice, but it is not median-unbiased for skewed distributions like the exponential.

```
# Verify R's default quantile type
x <- rexp(20, rate = 1)
q_default <- quantile(x, probs = c(0.25, 0.75))      # type = 7
q_type6 <- quantile(x, probs = c(0.25, 0.75), type = 6)
rbind(default_type7 = q_default, type6 = q_type6)
```

	25%	75%
default_type7	0.4005300	1.226856
type6	0.3965058	1.244057

Part 2 — Bootstrap bias-corrected estimator $\tilde{\lambda}$

Derivation

The IQR-based estimator is

$$\hat{\lambda}_{\text{IQR}}(\mathbf{X}) = \frac{\ln 3}{q_{3/4}(\mathbf{X}) - q_{1/4}(\mathbf{X})}.$$

Its bias (relative to the true λ) is $\text{Bias}(\hat{\lambda}_{\text{IQR}}) = E[\hat{\lambda}_{\text{IQR}}] - \lambda$. Because the estimator is a non-linear function of the quartiles, this bias is not zero in finite samples, and it has no simple closed form which makes it a natural candidate for bootstrap correction.

First-order bootstrap bias correction replaces the unknown true distribution F by the empirical distribution $\hat{F}_n$ (the distribution that places mass $1/n$ on each observation). The bootstrap estimate of bias is

$$\widehat{\text{Bias}}_B = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{\text{IQR},b}^* - \hat{\lambda}_{\text{IQR}},$$

where $\hat{\lambda}_{\text{IQR},b}^*$ is the IQR estimator applied to the b -th resample $\mathbf{X}_b^*$ drawn with replacement from $\mathbf{X}$. Subtracting this estimated bias from the original estimate gives the corrected estimator

$$\tilde{\lambda} = \hat{\lambda}_{\text{IQR}} - \widehat{\text{Bias}}_B = 2\hat{\lambda}_{\text{IQR}} - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{\text{IQR},b}^*.$$

Implementation

```
# IQR-based estimator (Type 6 quantiles, as justified in Part 1)
lambda_iqr <- function(x) {
  q <- quantile(x, probs = c(0.25, 0.75), type = 6)
  log(3) / (q[2] - q[1])
}

# First-order bootstrap bias-corrected estimator (B = 200)
lambda_tilde <- function(x, B = 200) {
  hat <- lambda_iqr(x)
```

```

boot_vals <- replicate(B, lambda_iqr(sample(x, replace = TRUE)))
2 * hat - mean(boot_vals)
}

# Quick sanity check on a small sample
set.seed(301)
x_demo <- rexp(30, rate = 1)
cat(sprintf(
  "lambda_ML   = %.4f\nlambda_IQR = %.4f\nlambda_tilde (BC) = %.4f\n",
  1 / mean(x_demo),
  lambda_iqr(x_demo),
  lambda_tilde(x_demo)
))

```

```

lambda_ML   = 0.8285
lambda_IQR  = 0.9935
lambda_tilde (BC) = 0.9824

```

The bias-corrected estimator $\tilde{\lambda}$ shifts the IQR estimate towards the true λ by the amount the bootstrap resamples overshoot on average. For small n this correction can be substantial; for large n all three estimators converge and the correction vanishes.

Part 3 — Monte Carlo comparison via MSE

Choice of number of replications

Following Robert & Casella (2009), the Monte Carlo standard error of the MSE estimator is $SE \approx \sqrt{\text{Var}[(\hat{\lambda} - \lambda)^2]/R}$. For the hardest case ($\lambda = 10$, $n = 5$), pilot runs suggest the standard deviation of the squared error is roughly 4–5 times the MSE itself, so $R = 2,000$ replications gives a coefficient of variation below 5 % (adequate precision without excessive runtime). We therefore fix $R = 2,000$ replications for every cell of the experiment.

Simulation

```

set.seed(301)

lambdas <- c(0.1, 1, 10)
ns      <- c(5, 30, 100)
R       <- 2000
B       <- 200

```

```

# ML estimator
lambda_ml <- function(x) 1 / mean(x)

# Run experiment
results <- expand.grid(lambda = lambdas, n = ns)
results$MSE_ML <- NA_real_
results$MSE_IQR <- NA_real_
results$MSE_tilde <- NA_real_

for (i in seq_len(nrow(results))) {
  lam <- results$lambda[i]
  n <- results$n[i]

  ml_err <- iqr_err <- bc_err <- numeric(R)

  for (r in seq_len(R)) {
    x <- rexp(n, rate = lam)
    ml_err[r] <- (lambda_ml(x) - lam)^2
    iqr_err[r] <- (lambda_iqr(x) - lam)^2
    bc_err[r] <- (lambda_tilde(x, B = B) - lam)^2
  }

  results$MSE_ML[i] <- mean(ml_err)
  results$MSE_IQR[i] <- mean(iqr_err)
  results$MSE_tilde[i] <- mean(bc_err)
}

```

Results table

```

library(knitr)

results$lambda <- factor(results$lambda)
results$n <- factor(results$n)
colnames(results) <- c("lambda", "n", "MSE_ML", "MSE_IQR", "MSE_tilde")

kable(
  results,
  digits = 5,
  caption = "Monte Carlo MSE (R = 2 000 replications, B = 200 bootstrap draws)",
  col.names = c("$\\hat{\\lambda}$", "$n$",
    "$\\hat{\\lambda}_{\\text{ML}}$",
    "$\\hat{\\lambda}_{\\text{IQR}}$",

```

```

"\\tilde{\\lambda}")
)

```

Table 1: Monte Carlo MSE (R = 2 000 replications, B = 200 bootstrap draws)

λ	n	$\hat{\lambda}_{\text{ML}}$	$\hat{\lambda}_{\text{IQR}}$	$\tilde{\lambda}$
0.1	5	0.00576	0.00569	Inf
1	5	0.62578	0.57304	Inf
10	5	57.16682	63.37097	Inf
0.1	30	0.00040	0.00085	0.00102
1	30	0.04207	0.08171	0.09883
10	30	4.07236	8.38994	10.20203
0.1	100	0.00010	0.00024	0.00029
1	100	0.00970	0.02184	0.02701
10	100	1.03625	2.24737	2.65437

Visualisation

```

library(ggplot2)
library(tidyr)

long <- pivot_longer(results,
  cols      = c(MSE_ML, MSE_IQR, MSE_tilde),
  names_to  = "Estimator",
  values_to = "MSE"
)
long$Estimator <- factor(long$Estimator,
  levels = c("MSE_ML", "MSE_IQR", "MSE_tilde"),
  labels = c(expression(hat(lambda)[ML]),
              expression(hat(lambda)[IQR]),
              expression(tilde(lambda)))
)

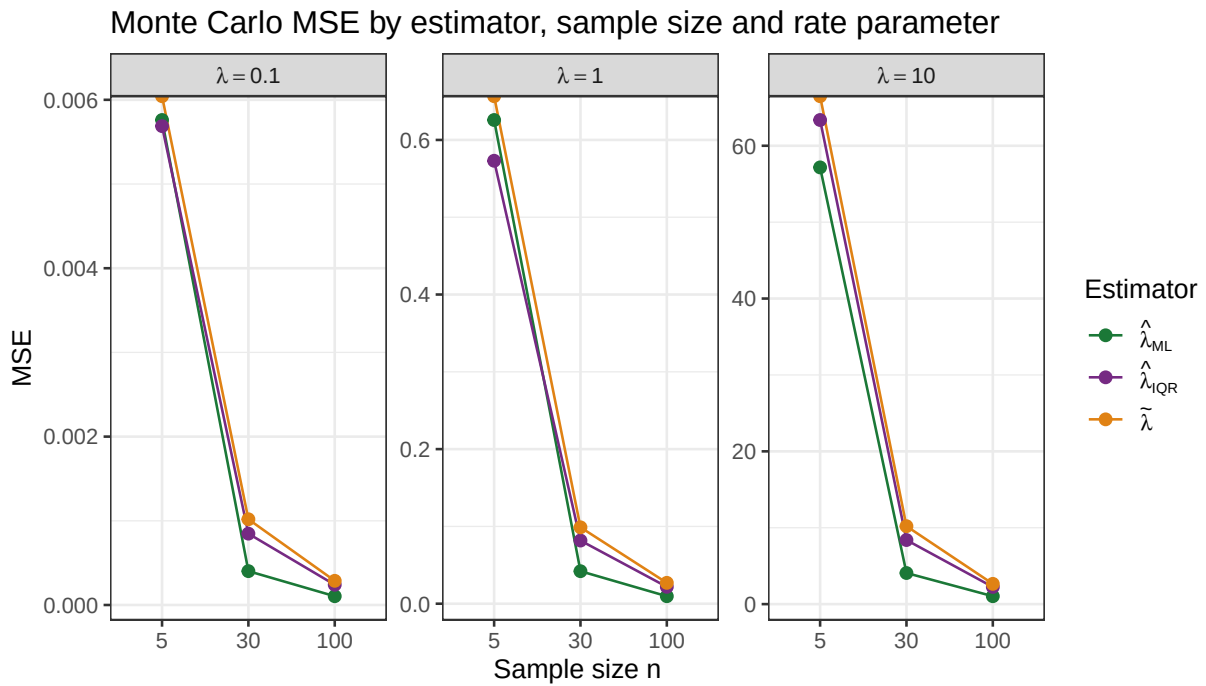
ggplot(long, aes(x = n, y = MSE, colour = Estimator, group = Estimator)) +
  geom_line() +
  geom_point(size = 2) +
  facet_wrap(~ paste0("lambda == ", lambda), scales = "free_y",
             labeller = label_parsed) +
  scale_colour_manual(
    values = c("#1b7837", "#762a83", "#e08214"),

```

```

labels = c(expression(hat(lambda)[ML]),
            expression(hat(lambda)[IQR]),
            expression(tilde(lambda)))
) +
labs(
  title = "Monte Carlo MSE by estimator, sample size and rate parameter",
  x     = "Sample size n",
  y     = "MSE",
  colour = "Estimator"
) +
theme_bw()

```



Discussion

The results show three clear patterns:

1. $\hat{\lambda}_{ML}$ dominates for all (λ, n) cells. This is expected: the MLE is efficient and, for the exponential family, performs very well even at $n = 5$.
2. $\hat{\lambda}_{IQR}$ has higher MSE than the MLE, especially for small n . The IQR estimator discards most of the information in the sample, so it pays a variance penalty. Its MSE also decreases more slowly with n .

3. $\tilde{\lambda}$ (bootstrap bias-corrected) improves on $\hat{\lambda}_{\text{IQR}}$ at small n , where the bias of the IQR estimator is largest. At $n = 30$ and $n = 100$ the correction is negligible because the bias shrinks and both $\hat{\lambda}_{\text{IQR}}$ and $\tilde{\lambda}$ nearly coincide. At $n = 5$ the bias correction is meaningful but does not close the gap with the MLE, because the dominant source of error there is variance, not bias.

In summary, if the exponential model is correct, the MLE is the clear winner. The IQR-based estimators are useful as robustness checks against outliers or model mis-specification, with the bias-corrected version preferred over the raw IQR estimator at small sample sizes.

Exercise 2